Video recording pack

Blast Radius — micro1 Agentic Workflows Hackathon, deliverable 3

The workflow

Record the terminal silently, then add the voice afterwards in CapCut. Decoupling the two means you never have to talk at the speed the terminal runs, and you can re-do a line without re-recording the footage. No webcam is needed: the whole video is one terminal window.

Step 1 — record the terminal

Open one terminal, maximised, in the repository root. Start your screen recorder, run the command below, and stop recording when the GitHub URL appears. That is about **4 minutes 50 seconds** of footage.

```
./demo.sh
```

Windows Game Bar (Win+G) is enough and needs no install. OBS gives you a tighter crop if you want one.

Legibility matters more than anything else here: dark theme, font size around 16–18pt so the text survives 1080p compression.

> **Do not run the demo commands by hand.** `demo.sh` writes only to `--mode demo` and deletes it afterwards. Running `run_oneshot.py --case 08` without `--mode` overwrites the committed baseline result and silently changes the numbers in your own changelog. That happened once already while this script was being checked.

Rehearse it first

Run the whole thing in about fifteen seconds to check every beat renders on your machine before you start recording:

```
SPEED=20 ./demo.sh
```

Step 2 — lay the voice over it

Drop the footage into CapCut and record the narration against it. Each beat overleaf names the blue banner you will see on screen, and its duration. Read the lines while that banner is up — every hold is sized for its narration with slack, so you are trimming rather than rushing. Word counts assume roughly 150 words per minute.

The voiceover script

Read each block against the blue banner named above it. Durations are how long that banner stays on screen.

A pull request against production infrastructure — 20s

This is a pull request against production infrastructure. It says: enable encryption at rest on the production database, to close a SOC 2 finding.

It's a security improvement. It's correct. And it's one line.

The entire change — 20s

That's the whole change. **`storage_encrypted`**, false to true.

Merging it destroys the database. That attribute is immutable on RDS, so Terraform satisfies the line by deleting the instance and creating a new empty one. Nothing in the diff says so.

Whoever approves this is the last checkpoint before production. This is exactly what they're there to catch.

The baseline: a static scanner, scored generously — 16s

The tool most teams already have is a static scanner. Here's Checkov across fifteen labelled pull requests — scored generously, restricted to the resources each change actually touches, which is better than it does in CI.

F1 of 0.32.

What the scanner says about case 08 — 18s

It does flag the database. For missing log exports.

The right resource, an entirely unrelated reason, and nothing at all about the deletion. Unscoped in real CI it reports nine point six findings on every pull request whatever changed, which is why nobody reads it.

The agent: one prompt, the diff, Haiku 4.5 on Bedrock — 12s

So here's the agent. One prompt — the diff, the pull request description, and instructions to report at most five findings and stay quiet about things that don't matter. Claude Haiku 4.5 through Bedrock. No plan, no tools, no retries.

The finding — 30s

Verdict: block. One finding, on the database, categorised **data-loss**.

“AWS does not support modifying storage_encrypted in place on an existing RDS instance. Terraform will destroy the current database and create a new one, resulting in data loss unless migrated via snapshot first. The PR description misrepresents this as a one-line configuration fix.”

Ten seconds, six tenths of a cent. And it got there from the diff alone — it knew that attribute is immutable without ever being shown the plan.

Iteration 1: add the terraform plan. Same case. — 40s

Which brings me to the thing this project was built around, and cut.

The premise was that a reviewer needs the plan, because the diff doesn't say what will happen. So iteration one adds it — same model, same prompt, same code path, one flag.

Same case. Still blocked. Still the right resource. Look at the category.

Without the plan: **data-loss** — it will destroy the database.

With the plan: **reliability** — the application will lose connectivity for ten to thirty minutes.

Downtime. Not data loss. Shown a plan that says *delete* then *create*, it reasoned about the resource lifecycle and stopped reasoning about what was inside it.

Across all fifteen cases — 14s

Across all fifteen cases: 0.78 without the plan, 0.67 with it. The plan is strictly more accurate than the diff, and it made the review worse. We removed it.

The same configuration, run four times, unchanged — 40s

The change that contributed most to this project didn't improve the agent at all. It's this.

Same configuration. Run four times. Unchanged.

0.90. 0.74. 0.76. 0.74.

For most of this project I reported 0.90 — and 0.90 was the best of four. I had already written up four separate iterations as regressions against it, each with a plausible mechanism, each committed.

Three of those four evaporated. They were inside the noise. My best story — that giving the agent a cost table made it stop reporting cost — died right here: one baseline run dropped that finding too, with no cost table anywhere.

Twelve minutes and thirty-three cents. It should have been the second measurement in this project, not the twenty-second.

Everything measured — 46s

So here's everything, with the three regressions re-run four times each so they're compared distribution against distribution.

Nothing beat one prompt. Three additions made it measurably worse; the other three couldn't be distinguished from doing nothing.

The worst was the most capable thing I built — an agent with tools, the plan, the scanner, unlimited steps. Nineteen times the cost, and it wandered: reviewing a load balancer change, it reported a finding about the database, because it could read the whole directory and did.

Two things I'd take forward.

Every context you add to fix a blind spot also tells the model that blind spot is someone else's job now. The plan made the review about resource lifecycles. The scanner made it about security. The cost table made the money look already accounted for.

And a single run is not a measurement. Structured outputs, fixed prompts, deterministic scoring — everything downstream of the model was reproducible, which quietly convinced me the model was too. Measure your noise floor first, and refuse to interpret any difference smaller than it.

github.com/Jamesokooboh/blast-radius	8s

Fifteen labelled cases, twenty-four configurations, about ten dollars end to end. The four-minute version is in the README.

Notes for the edit

- The strongest shot is the two case 08 findings on screen together, with **DATA-LOSS** and **RELIABILITY** both visible. It holds for forty seconds; use them.
- The four variance numbers print one at a time, three seconds apart. The beat lands on the fourth.
- Do not smooth over the retraction. A project that withdrew four of its own findings is more credible than one that reported eight.
- Model output is not deterministic. The live run in the footage will not match the committed file word for word, and may return a different number of findings — which is why the narration reads the committed result while the live run is only shown executing. If a take contradicts the committed result, say so rather than re-shooting until it agrees. That is the whole point of the variance section.
- If it runs long, cut the scanner section to one sentence over the 0.32. It is the only compressible part.

What the video has to cover

The brief asks for	Where it happens
The problem and simple baseline	Beats 1–4
One realistic execution, start to finish	Beats 5–6
One experiment you removed	Beats 7–8 — the terraform plan
The change that contributed most	Beat 9 — the variance check
The final comparison and changelog	Beat 10